

# UNIVERSIDAD POLITÉCNICA DE CHIAPAS

## **Alumnos:**

Luis Ángel Pérez Aguilera

"243757"

Angel Eduardo Chamé Pérez

"243770"

Luis Antonio Selvas de León

"243472"

## **Materia:**

*Bases de Datos Avanzadas*

**5B - ITleID**

A 05 de febrero del 2026

**Suchiapa, Chiapas.**

## Índice

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Preguntas:</b> .....                             | <b>3</b>  |
| <b>Entidades:</b> .....                             | <b>4</b>  |
| Principales:.....                                   | 4         |
| Dependientes.....                                   | 4         |
| Entidades de historial/control.....                 | 4         |
| Entidades asociativas.....                          | 5         |
| Entidades operativas.....                           | 5         |
| <b>Reportes (VIEWS).....</b>                        | <b>6</b>  |
| <b>Restricciones de Integridad Destacadas:.....</b> | <b>9</b>  |
| <b>Queries de ejemplo:.....</b>                     | <b>10</b> |

# Caso 3C

## Preguntas:

**1. ¿Por qué escogieron este caso?** "Analizamos las tres opciones y elegimos el **Caso 3-C** porque, aunque es de alta complejidad por la cantidad de tablas, tiene una lógica de negocio muy clara: el seguimiento de un ciudadano desde que mete su solicitud hasta que recibe el apoyo. Nos pareció el caso más robusto para aplicar un control estricto de auditoría (con la bitácora) y gestión de presupuestos, que es donde realmente se pone a prueba una base de datos de gobierno, sin meterle complicaciones innecesarias como logística de rutas o inventarios."

**2. ¿Por qué lo diseñaron así?** "Lo diseñamos pensando en que los datos nunca se pierdan y que los reportes sean exactos. Por ejemplo:

- Usamos una **entidad asociativa (Padrón)** para que una persona pueda estar en varios programas a la vez sin repetir sus datos.
- Implementamos una **entidad débil para la bitácora**, así guardamos el historial de quién aprobó o rechazó cada trámite y por qué, en lugar de sólo sobrescribir el estado actual.
- Estructuramos las tablas de **entregas** de forma que soporten tanto apoyos económicos como en especie (productos), para que el sistema sea flexible y resiste cualquier tipo de programa social futuro."

## Entidades:

### Principales:

#### -**beneficiarios**

Ciudadanos que solicitan y reciben los apoyos (identificados por CURP).

#### -**municipios**

Catálogo territorial para segmentar apoyos y calcular cobertura.

#### -**programas**

Representa cada programa social del gobierno (ej. Becas Futuro)

#### -**evaluadores**

Funcionarios encargados de revisar y dictaminar las solicitudes.

### Dependientes

#### -**convocatorias**

Dependen de un programa

#### -**solicitudes**

Intentos de un beneficiario por entrar a una convocatoria.

### Entidades de historial/control

#### -**bitacora\_estados**

Historial de cambios de estado de una solicitud.

## Entidades asociativas

### **-padron\_beneficiarios**

Relaciona beneficiarios con programas.

## Entidades operativas

### **-entregas**

Registro de apoyos entregados.

### **-entregas\_especie**

Detalle de productos entregados físicamente.

# Reportes (VIEWS)

- **Cobertura:** Beneficiarios y monto total por municipio y programa.

```
CREATE OR REPLACE VIEW vw_cobertura_municipio_programa AS
SELECT
    m.nombre AS municipio,
    p.nombre AS programa,
    COUNT(DISTINCT b.id) AS total_beneficiarios,
    COALESCE(SUM(e.monto_entregado), 0) AS monto_total_entregado,
    ROUND((COUNT(DISTINCT b.id)::NUMERIC / NULLIF(m.poblacion_total, 0)) * 100, 2) AS
    porcentaje_cobertura
FROM municipios m
JOIN beneficiarios b ON m.id = b.id_municipio
JOIN padron_beneficiarios pb ON b.id = pb.id_beneficiario
JOIN programas p ON pb.id_programa = p.id
LEFT JOIN entregas e ON b.id = e.id_beneficiario AND p.id = e.id_programa
GROUP BY m.nombre, p.nombre, m.poblacion_total;
```

- **Eficiencia:** Tasa de aprobación y rechazo por convocatoria.

```
CREATE OR REPLACE VIEW vw_taza_aprobacion_rechazo_convocatoria AS
SELECT
    c.id AS ID_convocatoria,
    p.nombre AS programa,
    COUNT(s.id) AS total_solicitudes,
    SUM(CASE WHEN s.estado = 'Aprobada' THEN 1 ELSE 0 END) AS aprobadas,
    SUM(CASE WHEN s.estado = 'Rechazada' THEN 1 ELSE 0 END) AS rechazadas,
    ROUND(
        (SUM(CASE WHEN s.estado = 'Aprobada' THEN 1 ELSE 0 END)::NUMERIC * 100) / NULLIF(COUNT(s.id),
        0), 2
    ) AS tasa_aprobacion
FROM programas p
JOIN convocatorias c ON p.id = c.id_programa
LEFT JOIN solicitudes s ON c.id = s.id_convocatoria
GROUP BY c.id, p.nombre;
```

- **Presupuesto:** Comparativa de gasto acumulado vs. presupuesto anual, clasificando programas sobrepasados o en subejecución.

```
CREATE OR REPLACE VIEW vw_gasto_vs_presupuesto AS
SELECT
  p.nombre AS programa,
  p.presupuesto_anual,
  COALESCE(SUM(e.monto_entregado), 0) AS gasto_anual_actual,
  ROUND(COALESCE(SUM(e.monto_entregado), 0) / NULLIF(p.presupuesto_anual, 0) * 100, 2) AS
  porcentaje_gastado,
  p.presupuesto_anual - COALESCE(SUM(e.monto_entregado), 0) AS saldo,
  CASE
    WHEN COALESCE(SUM(e.monto_entregado), 0) / NULLIF(p.presupuesto_anual, 0) * 100 <= 70 THEN 'En
curso'
    WHEN COALESCE(SUM(e.monto_entregado), 0) / NULLIF(p.presupuesto_anual, 0) * 100 BETWEEN 70 AND
100 THEN 'Cerca del limite'
    ELSE 'Sobrepasado'
  END AS clasificacion
FROM programas p
LEFT JOIN entregas e ON p.id = e.id_programa
  AND e.fecha_entrega >= DATE '2026-01-01'
  AND e.fecha_entrega < DATE '2027-01-01'
GROUP BY p.id, p.nombre, p.presupuesto_anual
ORDER BY porcentaje_gastado DESC;
```

- **Multi-beneficiarios:** Personas activas en más de un programa a la vez.

```
CREATE VIEW vw_beneficiarios_multiplos_programas AS
SELECT
  b.id AS beneficiario_id,
  b.curp,
  b.nombre,
  b.apellido_paterno,
  COUNT(DISTINCT pb.programa_id) AS total_programas_activos,
  COALESCE(SUM(e.monto_entregado), 0) AS
  monto_beneficiario
FROM beneficiarios b
JOIN padron_beneficiarios pb
  ON pb.beneficiario_id = b.id
  AND pb.estatus_padrón = 'Activo'
LEFT JOIN entregas e
  ON e.beneficiario_id = b.id
  AND e.programa_id = pb.programa_id
GROUP BY b.id, b.curp, b.nombre, b.apellido_paterno
HAVING COUNT(DISTINCT pb.programa_id) > 1;
```

- **Bitácora de rechazos:** Historial de solicitudes denegadas en los últimos 90 días.

```
CREATE VIEW vw_rechazos_recientes AS
SELECT
    s.id AS solicitud_id,
    b.curp,
    b.nombre,
    be.fecha_cambio AS fecha_rechazo,
    COALESCE(be.motivo, 'Sin motivo registrado') AS
FROM vbitacora_estados be
JOIN solicitudes s
    ON s.id = be.solicitud_id
JOIN beneficiarios b
    ON b.id = s.beneficiario_id
WHERE
    be.estado_nuevo = 'Rechazada'
    AND be.fecha_cambio >= CURRENT_DATE - INTERVAL '90 days';
```



## Restricciones de Integridad Destacadas:

1. **UNIQUE (CURP):** Evita que un ciudadano se registre dos veces con identidades duplicadas.
2. **CHECK (Presupuesto > 0):** Asegura que no existan programas con presupuesto negativo.
3. **CHECK (Estados):** Restringe la columna `estado` a los valores permitidos ('Recibida', 'En revisión', etc.).
4. **NOT NULL (Nombres/Fechas):** Campos obligatorios para la validez de las convocatorias.
5. **FK (Integridad Referencial):** Garantiza que no se registre una entrega a un beneficiario que no existe en la tabla maestra.
6. **CHECK (Fechas):** La `fecha_fin` de una convocatoria debe ser mayor a la `fecha_inicio`.

## Queries de ejemplo:

- **Consulta con JOIN y Agregación:** Para saber cuánto dinero en total ha recibido un beneficiario específico sumando todos sus programas.

```
SELECT
    b.nombre,
    b.apellido_paterno,
    COUNT(e.id) AS total_entregas,
    SUM(e.monto_entregado) AS monto_acumulado
FROM beneficiarios b
JOIN entregas e ON b.id = e.id_beneficiario
GROUP BY b.id, b.nombre, b.apellido_paterno
HAVING SUM(e.monto_entregado) > 0;
```

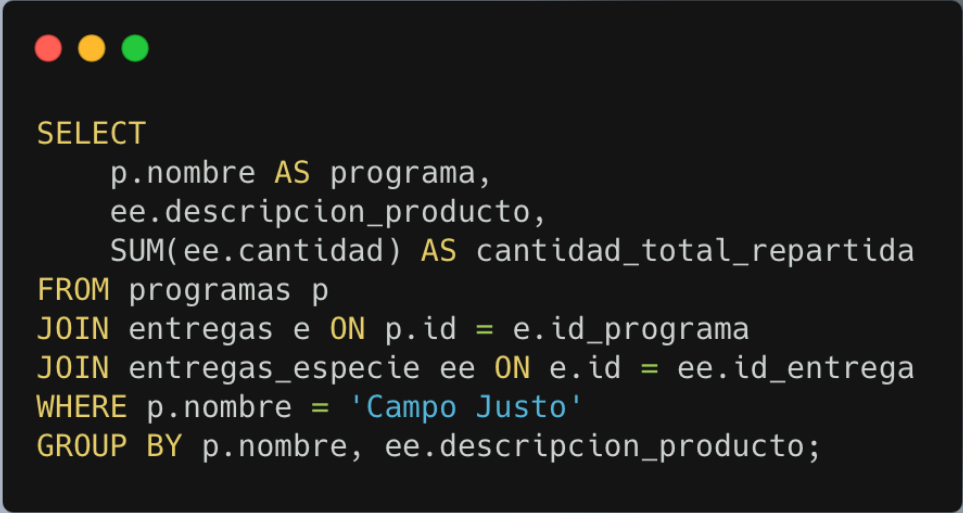
- **Consulta con CASE y Subconsultas:** Para listar solicitudes mostrando si el evaluador no ha sido asignado.

```
SELECT
  s.id AS folio_solicitud,
  b.curp,
  COALESCE(ev.nombre, 'PENDIENTE DE ASIGNAR') AS evaluador_responsable,
  CASE
    WHEN s.estado = 'Aprobada' THEN 'Trámite Finalizado con Éxito'
    WHEN s.estado = 'Rechazada' THEN 'Trámite Finalizado - No Elegible'
    ELSE 'En Proceso Administrativo'
  END AS estatus_informativo
FROM solicitudes s
JOIN beneficiarios b ON s.id_beneficiario = b.id
LEFT JOIN evaluadores ev ON s.id_evaluador_asignado = ev.id;
```

- **Consulta de Historial:** Para ver el recorrido de una solicitud específica para ver quién cambió su estado y por qué.

```
SELECT
  be.fecha_cambio,
  be.estado_anterior,
  be.estado_nuevo,
  ev.nombre AS realizado_por,
  COALESCE(be.motivo, 'No se registró motivo') AS observacion
FROM bitacora_estados be
JOIN evaluadores ev ON be.id_usuario_cambio = ev.id
WHERE be.id_solicitud = 1
ORDER BY be.fecha_cambio DESC;
```

- **Consulta de "Campo Justo":** Para mostrar qué productos se entregaron en un programa específico que maneja apoyos físicos.



```
SELECT
    p.nombre AS programa,
    ee.descripcion_producto,
    SUM(ee.cantidad) AS cantidad_total_repartida
FROM programas p
JOIN entregas e ON p.id = e.id_programa
JOIN entregas_especie ee ON e.id = ee.id_entrega
WHERE p.nombre = 'Campo Justo'
GROUP BY p.nombre, ee.descripcion_producto;
```